MixBytes()

[SECURITY AUDIT REPORT]

Resolv Vault Street

PREPARED FOR RESOLV

REPORT DATE

August 31, 2026

[CONTENTS]

Table of Contents

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

Vault Street is a product-agnostic framework for issuing **permissioned, asset-backed ERC-20 tokens**. Each on-chain product (e.g. `primeUSD`) is a configured deployment of these contracts.

A product consists of a **share token** (the issued ERC-20, transfer-restricted to a whitelist), a **deposit token** (the backing collateral, e.g. USDC), and a **request manager** that mints shares against deposits and burns shares to release deposits, gated by an oracle-derived price and an off-chain completer.

The mint/burn flow is **asynchronous**: providers create requests on-chain; a privileged off-chain completer assigns the precise share/deposit amount within a configured deviation tolerance.

In scope are an upgradeable six-decimal ERC20 `PermissionedToken` whose transfers are gated by an `AddressesWhitelist`, minting constrained by an optional `MintGuard`, an admin-managed on-chain price feed `PriceStorage`, and an immutable `ExternalRequestsManager` that lets whitelisted providers asynchronously request mints and burns of deposit collateral against issued shares, with privileged off-chain completers finalizing requests against oracle prices within configurable deviation bounds.

The security review was conducted over 3 working days via manual code review and a proprietary AI-assisted analysis tool, followed by a re-audit of client remediations.

The review examined both the in-scope contract logic and how those components interact at runtime with configured ERC20 assets, the treasury, and the oracle interface. We also applied MixBytes' standard internal checklist covering access control, reentrancy, accounting and rounding, idempotency, pause semantics, and privileged configuration paths.

Off-chain trust roots — the price-setter service, mint/burn completer operators, and the treasury Safe/EOA — were out of scope. ERC20 implementations other than PermissionedToken configured as baseAsset or quoteAsset are external to the scoped source files and were not subject to line-by-line source review. Mainnet deployment verification did review the configured asset addresses and constructor/initializer arguments.

Key notes:

- `CarryPriceStorageAggregatorV3Adapter` currently does not enforce price staleness; this is consistent with the official `MorphoChainlinkOracleV2`, which relies on the underlying feed's freshness guarantees to preserve market liveness, including liquidations, but it also introduces a risk of stale prices being used indefinitely, whereas other Morpho integrations, such as the Pyth wrapper, enforce a maximum price age at the cost of potential oracle reverts.
- In some cases the mint guard can allow minting up to two window limits — for example at the boundary of two consecutive windows (the window is discrete, not sliding), and when rotating via `PermissionedToken.setMintGuard()` (guard rotation resets rate limits). However, in the current implementation this is not critical, since an attack would bypass at most one full rate limit. For example, with a mint guard of 1% TVL per hour, if an attacker steals minter and burner keys to mint slightly above the fair amount within deviation and immediately burn slightly below, they could mint ~2.02% of TVL across two window boundaries. **This is intended;** limits should be configured to account for this nuance.
- In `preMint()`, `effectiveLimit` depends on `totalSupply`, but minting increases `totalSupply`, which expands `effectiveLimit`. For example, with `bpsLimit = 10%` and `totalSupply = 100M`, one bucket can mint ~11M rather than 10M. **This is intended:** this is a convergent series with a fixed ceiling.

- **Escrowed shares trapped on provider de-whitelist or token pause**. If a provider opens a burn request and is later removed from the token transfer whitelist, `cancelBurn()` reverts. However, the token is permissioned, so this is not an issue.
- **PriceStorage update window vs staleness:** After each price publish at time `T`, the next `setPrice()` is allowed only once `block.timestamp >= T + minUpdateInterval`, while `isStale()` becomes true when `block.timestamp > T + maxStaleness`. The price setter must therefore land every refresh in the interval `(T + minUpdateInterval, T + maxStaleness]` (on-chain only requires `minUpdateInterval < maxStaleness`). If `maxStaleness - minUpdateInterval` is too tight relative to off-chain latency (bot interval, retries, step-bound rejections, idempotency key handling), the feed can go stale even with a diligent operator — blocking `completeMint()` / `completeBurn()` and `MintGuard.preMint()`. We recommend configuring `maxStaleness` with a comfortable margin above `minUpdateInterval` (and above the real-world update cadence), not the minimal gap that passes initialization checks.
- `PermissionedToken` `MINTER_ROLE` **and** `BURNER_ROLE` **should either be** `ExternalRequestsManager` **or an extremely well-guarded multisig/DAO:** In the standard deployment, both roles are granted to `ExternalRequestsManager` at initialization; alternate holders should only be considered in non-standard operational scenarios.
- `PermissionedToken.initialize` / `PriceStorage.initialize` can be frontrun in a non-atomic deployment process. **We recommend using atomic proxy deployment.**
- `DEFAULT_ADMIN_ROLE` **should be an extremely well-guarded multisig or DAO.** In case of compromise, it can drain all funds from the protocol.
- `CONFIG_ROLE` / `ISSUANCE_CONFIG_ROLE` **should be a well-guarded multisig or DAO.** Compromise of it (esp. together with `PRICE_SETTER_ROLE`) would allow bypassing issuance constraints.
- `PRICE_SETTER_ROLE` **should also be extremely well-guarded.**
- Although ERM `cancelBurn()` deliberately has no the `whenNotPaused` modifier, pausing the `PermissionedToken` will still block `cancelBurn()`, since the refund transfer goes through the token's `_update()` hook, which reverts while paused.
- **Oracle migration nuances.** `PriceStorage.getPrice()` never reverts, but if another oracle whose `getPrice()` can revert is used, `setOracle()` will may break. Also, when migrating to a new oracle while the old one's price is stale and far from the market, the new oracle must initially be seeded near the old price (within deviation) before moving to the new level. **This is intended.**
- `completeMint()` / `completeBurn()` **MEV risk:** Completion transactions are built off-chain, presumably against the oracle price the operator expects at broadcast time. If a `PriceStorage.setPrice()` update is visible in the public mempool, an attacker can bundle their own trade with the `setPrice()` transaction to execute close to when the off-chain server builds and submits the completion, which may allow the attacker to extract value from the protocol. **We recommend sending oracle price updates through a private mempool.**
- `Timelock` **executor address validation.** `Timelock` only checks that the `_executors` array is non-empty, not its contents. Passing `address(0)` as an executor grants `EXECUTOR_ROLE` to the zero address, which the inherited `onlyRoleOrOpenRole` treats as an open role — making `execute()` / `executeBatch()` callable by anyone.

1.3 Project Overview

Summary

CLIENT	Resolv
CATEGORY	RWA
PROJECT	Vault Street
TYPE	Solidity
PLATFORM	EVM
TIMELINE	21.05.2026 - 31.08.2026

Scope of Audit

valiant/src/AddressesWhitelist.sol
valiant/src/ExternalRequestsManager.sol
valiant/src/MintGuard.sol
valiant/src/PermissionedToken.sol
valiant/src/PriceStorage.sol
valiant/src/interfaces/IAddressesWhitelist.sol
valiant/src/interfaces/IExternalRequestsManager.sol
valiant/src/interfaces/IMintGuard.sol
valiant/src/interfaces/IOracle.sol
valiant/src/interfaces/IPermissionedToken.sol
valiant/src/interfaces/IPriceStorage.sol
valiant/src/interfaces/IPriceStorageAggregatorV3Adapter.sol
valiant/src/CarryPriceStorageAggregatorV3Adapter.sol
valiant/src/Timelock.sol

Versions Log

DATE	COMMIT HASH	NOTE
21.05.2026	5cf7b0b8890c9015ba980f86635e5a4e3a10576f	Initial Commit
01.06.2026	a7f20d7cb8bc8ea7668114539dcba25813e21da9	Commit for re-audit
10.06.2026	55773c99dd2c0e64be4378923c63672fd7a92250	Commit for re-audit
21.08.2026	7b84a60f4fc44e2926172a7eae4ca20c0c5d11f1	Scope extension (CarryPriceStorageAggregatorV3Adapter.sol)
31.08.2026	7f9b911426b3c545cd55e68cb4d0026c5866fc28	Scope extension (Timelock.sol)

Mainnet Deployments

FILE	ADDRESS	BLOCKCHAIN
AddressesWhitelist.sol	0xf436F5AD67D4dC60446420E83E22B7B5639A4f6c	Ethereum
PermissionedToken.sol (proxy, primeUSD)	0x7Ea76108975EC0998B9bc2db04B4ECa986400DD7	Ethereum
PermissionedToken.sol (impl)	0xb0b01a72b6d32142BB6794A7033769F0e2FCF493	Ethereum
ProxyAdmin.sol (Token)	0x3257339553fff27B311eE1fEBD7FA6bB77385056	Ethereum
PriceStorage.sol (proxy)	0x8CDa03E2004c35e07963fb792C6b7511DabEE369	Ethereum
PriceStorage.sol (impl)	0x03e0116B73F493A6aB6C1a5066F30A11350B9cFb	Ethereum
ProxyAdmin.sol (Oracle)	0x3FC12e4773b10B0d0D5e02De042261868e10852D	Ethereum
MintGuard.sol	0x5501AA12F419B325a46ee8c7d6b2D89FD62da289	Ethereum
ExternalRequestsManager.sol	0x8C14B6e8EC9968cd9c69EEdeA4A1295AEc5E5D6e	Ethereum
CarryPriceStorageAggregatorV3Adapter.sol	0x90a30aC2C59E7376FaE7977cC14518593Cbd1B8C	Ethereum
PriceStorage.sol (proxy, CARRY)	0xd610FAbAB31C6D76B50A49C337fc39D6559E0E87	Ethereum
PriceStorage.sol (impl, CARRY)	0x9a56d6604522432dcaF08417ab030797822Ca3ee	Ethereum
Timelock.sol	0x61c2EB9701D70c44A9d910C06743cc25a0156B96	Ethereum

The core Vault Street deployment was reviewed against 55773c99dd2c0e64be4378923c63672fd7a92250, the `CarryPriceStorageAggregatorV3Adapter` scope extension against 7b84a60f4fc44e2926172a7eae4ca20c0c5d11f1 (on-chain state read at block 25824987), and `Timelock` against 7f9b911426b3c545cd55e68cb4d0026c5866fc28 (block 25874918).

Verified on-chain source for `Timelock` matches commit `7f9b911` byte-for-byte; it is a plain (non-proxy) contract with `getMinDelay() = 2 days`, self-administered, with proposer/executor/canceller roles on the admin Safe (`0x14d7d788f9C71A2cDB851DF884fB908050DFb222` , 3-of-5) and an additional EOA proposer/canceller (`0x3f6bc3b1f1c00b8c8648f66e41069fd4ca372097`).

Verified on-chain source for `AddressesWhitelist` , `PermissionedToken` (impl), `PriceStorage` (impl), `MintGuard` , and `ExternalRequestsManager` matches the local repository byte-for-byte (including OpenZeppelin dependencies). Proxy contracts (`PermissionedToken` , `PriceStorage`) and both `ProxyAdmin` instances are standard OpenZeppelin `TransparentUpgradeableProxy` / `ProxyAdmin` and also match the vendored OZ sources. ERC-1967 implementation slots resolve as expected: the primeUSD token proxy points to `0xb0b01a72b6d32142BB6794A7033769F0e2FCF493` and the PriceStorage proxy points to `0x03e0116B73F493A6aB6C1a5066F30A11350B9cFb` .

Configured asset wiring was verified on-chain. `PriceStorage` reports `baseAsset` = primeUSD proxy (`0x7Ea76108975EC0998B9bc2db04B4ECa986400DD7` , symbol `primeUSD` , 6 decimals) and `quoteAsset` = Circle USDC (`0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48` , `USDC` , 6 decimals), with `priceDecimals = 8` . `ExternalRequestsManager` resolves `issueToken` / `depositToken` to the same addresses via immutable

constructor wiring from `oracle.baseAsset()` / `oracle.quoteAsset()`, with `oracle` pointing to the `PriceStorage` proxy. `MintGuard` references the same `token` and `oracle`. The `baseAsset` address is the audited `PermissionedToken` deployment; `quoteAsset` is the canonical Ethereum mainnet USDC contract.

Admin ownership is split across two Gnosis Safe proxies (singleton `0x41675C099F32341bf84BFc5382aF534df5C7461a`, Safe v1.4.1). The admin Safe (`0x14d7d788f9C71A2cDB851DF884fB908050DFb222`) owns the primeUSD proxy, PriceStorage proxy, both ProxyAdmins, `MintGuard`, and `ExternalRequestsManager`; it uses threshold `3` with `5` EOA owners. `AddressesWhitelist` is owned separately by Safe `0x7bb70aa3762Bcfd5C6980A16b44627eF84f27916` with threshold `2` and `4` EOA owners. All listed owners are EOAs; no nested Safes or controller contracts were observed on the ownership paths checked.

The `CarryPriceStorageAggregatorV3Adapter` (`0x90a30aC2C59E7376FaE7977cC14518593Cbd1B8C`) was added as a scope extension and is deployed as a plain (non-proxy) immutable contract; its verified on-chain source, together with its dependencies (`IPriceStorage`, `IPriceStorageAggregatorV3Adapter`, and OpenZeppelin `SafeCast`), matches commit `7b84a60f` byte-for-byte. The adapter exposes a CARRY `PriceStorage` feed through the Chainlink `AggregatorV3Interface` shape and reports `decimals() = 8`, `version() = 87`, and `description() = "CARRY Price AggregatorV3 interface"`. It has no owner or privileged roles; its only configuration is the immutable `priceStorage` pointer, which resolves to `0xd610FAbAB31C6D76B50A49C337fc39D6559E0E87`, the CARRY `PriceStorage` proxy that reports `priceDecimals() = 8` and a non-zero last price. At the reviewed block, `latestRoundData()` returns the same value/timestamp as the underlying feed's `lastPrice()` (`roundId` and `answeredInRound` are `0` by design, since PriceStorage does not track round ids), confirming the adapter is correctly wired to the live CARRY feed.

The CARRY `PriceStorage` behind the adapter was also verified. The proxy at `0xd610FAbAB31C6D76B50A49C337fc39D6559E0E87` is a standard OpenZeppelin `TransparentUpgradeableProxy` whose verified source (13 OZ files including `TransparentUpgradeableProxy`, `ProxyAdmin`, `ERC1967Utils`) matches the vendored OZ sources byte-for-byte. Its ERC-1967 implementation slot points to `0x9a56d6604522432dcaF08417ab030797822Ca3ee`, a `PriceStorage` implementation whose verified source (`PriceStorage.sol`, `IPriceStorage.sol`, and its OZ / OZ-upgradeable dependencies) matches commit `7b84a60f` byte-for-byte. Upgrades are gated by a `ProxyAdmin` (`0x054973E48158BC2304f17BB83ad125A3e390fDB2`) owned by the same 3-of-5 admin Safe (`0x14d7d788f9C71A2cDB851DF884fB908050DFb222`), which also holds `defaultAdmin()` on the CARRY `PriceStorage`. The feed prices CARRY against USDC: `baseAsset` resolves to the CARRY token (`0xF05F7Ab9B05D9Dcf99B8E9bBAE8E5e4A3201D004`, symbol `CARRY`, 6 decimals) and `quoteAsset` to canonical Ethereum mainnet USDC (`0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48`).

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2 Re-audit & Mainnet Verification

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

MAINNET DEPLOYMENT VERIFICATION

OBJECTIVE: FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

MEDIUM - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

LOW - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

LIKELIHOOD

HIGH - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

MEDIUM - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

LOW - A highly unlikely event that can only be triggered by the contract owner.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

● FIXED - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● PARTIALLY FIXED - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● ACKNOWLEDGED - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

1.6 Summary of Findings

Findings Count

0	0	0	9	9
CRITICAL	HIGH	MEDIUM	LOW	TOTAL

Findings Statuses

All finding statuses reflect the state of the code at the latest revision listed in the Versions Log.

ID	FINDING	SEVERITY	STATUS
L-1	Inconsistent PriceStorage bounds can block all new price updates	LOW	FIXED
L-2	Large mint requests can be uncompletable under MintGuard per-block cap	LOW	ACKNOWLEDGED
L-3	Mint and burn requests have no on-chain expiry	LOW	ACKNOWLEDGED
L-4	ExternalRequestsManager.setPriceDeviationBps() — upper bound of 100% allows settlement at [0, 2×actual]	LOW	ACKNOWLEDGED
L-5	PriceStorage.initialize() missing minUpdateInterval < maxStaleness cross-check	LOW	FIXED
L-6	PriceStorage.initialize() — initialPrice == 0 permanently bricks subsequent setPrice via band collapse at zero	LOW	FIXED
L-7	ExternalRequestsManager.setOracle() — rotation without probe bricks settlements until new oracle is seeded	LOW	FIXED
L-8	Insufficient protection against compromised MINT_COMPLETER_ROLE / BURN_COMPLETER_ROLE	LOW	FIXED
L-9	Oracle rotation — no on-chain check that new price is within an acceptable band of the old oracle	LOW	FIXED

2.1 Critical

NO CRITICAL SEVERITY FINDINGS

2.2 High

NO HIGH SEVERITY FINDINGS

2.3 Medium

NO MEDIUM SEVERITY FINDINGS

2.4 Low

L-1 Inconsistent PriceStorage bounds can block all new price updates

LOW

FIXED

DESCRIPTION

`PriceStorage._setPrice()` applies two independent constraints on every new price: `minPrice <= x <= maxPrice` and `lowerBoundPercentage` / `upperBoundPercentage` derived from the last price.

`setMinPrice()` and `setMaxPrice()` are callable by `CONFIG_ROLE` and only require `minPrice < maxPrice`. They do not check that `minPrice <= lastPrice <= maxPrice`.

A privileged admin due to human error can configure, for example, `minPrice < maxPrice << lastPrice.value`. Every subsequent `setPrice()` must pick `_price <= maxPrice`, but the step-bound logic still anchors on how much the new price can be lowered; so the lowest new price may still be greater than `maxPrice`, making the new price unviable, and the oracle stale.

RECOMMENDATION

> We recommend enforcing consistency with the last price in `setMinPrice()` / `setMaxPrice()`.

CLIENT'S COMMENTARY

Fixed. `13fadf8`
`setMinPrice` and `setMaxPrice` now also revert if the new bound would put `lastPrice.value` outside `[minPrice, maxPrice]` (`MinPriceAboveLastPrice` / `MaxPriceBelowLastPrice`).

L-2 Large mint requests can be uncompletable under MintGuard per-block cap

LOW

ACKNOWLEDGED

DESCRIPTION

`ExternalRequestsManager.requestMint()` accepts any deposit above `minMintRequestAmount` and does not check `MintGuard.mintLimitPerBlock`. When `completeMint()` runs, it calls `PermissionedToken.mint()`, which invokes `MintGuard.preMint(_sharesToMint, totalSupply())`. The guard requires `mintedPerBlock[block.number] + _amount <= mintLimitPerBlock` in a **single** transaction — there is no way to split one request across blocks.

If the oracle-implied share amount for the escrowed deposit (or the completer's chosen `_sharesToMint` that still satisfies `priceDeviationBps` against `actualShares`) exceeds `mintLimitPerBlock`, every `completeMint()` attempt reverts with `MintRateLimitExceeded`. The request stays in `CREATED` with `depositToken` locked on the manager until the provider calls `cancelMint()` or parameters change. This is especially likely when `minExpectedAmount` or the fair mint size is above the per-block limit.

RECOMMENDATION

> We recommend rejecting mint requests that exceed `mintLimitPerBlock`.

CLIENT'S COMMENTARY

Acknowledged
Even if the user has already requested the mint, the operator may still get stuck at the completion step due to mint guard limits.
Since mint completion is fully manual through the multisig and is not even quasi-instant, this flow could create unnecessary operational issues.

L-3 Mint and burn requests have no on-chain expiry

LOW

ACKNOWLEDGED

DESCRIPTION

`requestMint()` and `requestBurn()` let the provider set `minExpectedAmount` (minimum shares to mint or minimum deposit to withdraw) but store **no deadline**. A request can remain in `CREATED` indefinitely with `depositToken` or `issueToken` escrowed on `ExternalRequestsManager`.

`completeMint()` / `completeBurn()` price the fill using the **current** oracle at completion time, not the price when the request was opened. If the oracle moves against the provider while the request is pending, the completer can still finalize within `priceDeviationBps` and `minExpectedAmount` — capturing positive slippage for the protocol/treasury (fewer shares minted per deposit, or less deposit returned per burn) while honoring the provider's floor.

RECOMMENDATION

> We recommend adding an on-chain expiry (e.g. `deadline` timestamp or max lifetime) after which only the provider can `cancelMint()` / `cancelBurn()`, or completion must use pricing rules tied to request creation time.

CLIENT'S COMMENTARY

Acknowledged
Since the product is permissioned and completion is fully manual through the multisig (not quasi-instant), an on-chain deadline would create operational friction and degrade UX without adding meaningful protection. Providers retain their own settlement floor via `minExpectedAmount` and can `cancelMint` / `cancelBurn` at any time before completion.

L-4 ExternalRequestsManager.setPriceDeviationBps() — upper bound of 100% allows settlement at [0, 2×actual]

LOW

ACKNOWLEDGED

DESCRIPTION

On completion, the completer chooses `_sharesToMint` / `_depositToReturn`. On-chain checks require that the chosen amount lies within `priceDeviationBps` of the oracle-derived `actualShares` / `actualDeposit` (`_assertWithinDeviation`).

`setPriceDeviationBps()` (`ISSUANCE_CONFIG_ROLE`) allows any value from `1` up to `10_000` bps (100%). At the maximum, the completer can pick an amount up to **twice** the oracle-derived figure (or near zero on the other side), subject only to `minExpectedAmount`.

If `ISSUANCE_CONFIG_ROLE` or `MINT_COMPLETER_ROLE` / `BURN_COMPLETER_ROLE` is compromised, governance can widen deviation to 100% and maximize extractable spread versus a low provider floor.

RECOMMENDATION

> We recommend adding an on-chain upper bound on `priceDeviationBps` well below 100% (e.g. tens of bps). Keep operational completer keys and `ISSUANCE_CONFIG_ROLE` on multisig; document that `minExpectedAmount` must reflect the intended worst-case fill.

CLIENT'S COMMENTARY

Acknowledged. The value will be set within reasonable bounds at deployment.

L-5 PriceStorage.initialize() missing minUpdateInterval < maxStaleness cross-check

LOW

FIXED

DESCRIPTION

`PriceStorage` 's public setters enforce the coherence invariant `minUpdateInterval < maxStaleness` : `setMaxStaleness()` requires `minUpdateInterval < _newMaxStaleness` ; `setMinUpdateInterval()` requires `_interval < maxStaleness` . `initialize()` does not enforce this cross-check between `p.minUpdateInterval` and `p.maxStaleness` .

With `maxStaleness = 60` and `minUpdateInterval = 3600` (each valid in isolation), `initialize()` succeeds. After 60 seconds, `isStale()` returns true, but the next price write is blocked until `block.timestamp >= lastPrice.timestamp + minUpdateInterval` (+3600s) — always after the stale deadline. Every `completeMint()` , `completeBurn()` , and `MintGuard.preMint()` then reverts with `OracleStale()` .

RECOMMENDATION

> We recommend adding `require(p.minUpdateInterval < p.maxStaleness, InvalidParams())` in `PriceStorage.initialize()` immediately after `_setMaxStaleness` / `_setMinUpdateInterval` , mirroring the public setters, so a misconfigured deployment reverts at init rather than shipping a permanently stale oracle.

CLIENT'S COMMENTARY

Fixed — `13fadf8`
`initialize` now reverts with `MaxStalenessBelowMinUpdateInterval` if the invariant is violated, mirroring the existing runtime setters.

L-6 PriceStorage.initialize() — initialPrice == 0 permanently bricks subsequent setPrice via band collapse at zero

LOW

FIXED

DESCRIPTION

`PriceStorage.initialize()` enforces that `initialPrice` lies in `[minPrice, maxPrice]` but does not require `initialPrice != 0`. If the operator passes `minPrice == 0` and `initialPrice == 0`, initialization succeeds. Internal `_setPrice(bytes32(0), 0)` sets `lastPrice.value = 0`.

Later `setPrice()` calls compute the step band around `lastPrice.value` as `upperBound = prevPrice + (prevPrice * upperBoundPercentage) / BOUND_PERCENTAGE_DENOMINATOR` and `lowerBound = prevPrice - (prevPrice * lowerBoundPercentage) / BOUND_PERCENTAGE_DENOMINATOR`. With `prevPrice == 0`, both bounds are `0`, so any non-zero candidate reverts with `InvalidPriceRange`. There is no setter that resets `lastPrice` directly; recovery requires an implementation upgrade.

RECOMMENDATION

> We recommend requiring `p.initialPrice != 0` and `p.minPrice != 0` in `initialize()`.

CLIENT'S COMMENTARY

Fixed — `13fadf8`
`_setMinPrice` rejects `0` (`InvalidMinPrice`), which transitively forces `initialPrice >= minPrice > 0` and eliminates the band-collapse-at-zero scenario.

L-7 ExternalRequestsManager.setOracle() — rotation without probe bricks settlements until new oracle is seeded

LOW

FIXED

DESCRIPTION

`ExternalRequestsManager.setOracle(_newOracle)` enforces `baseAsset`, `quoteAsset`, and `shareScale` compatibility but does not call `!_newOracle.isStale()` or check that `_newOracle.getPrice() != 0`. Rotating to a structurally compatible but un-seeded oracle bricks all settlement paths until the new oracle is seeded.

RECOMMENDATION

> We recommend requiring `!oracle.isStale()` and `oracle.getPrice() != 0` after the new pointer is written in `setOracle`.

CLIENT'S COMMENTARY

Fixed — `2543fb2`

Fixed in both `ExternalRequestsManager.setOracle` and `MintGuard.setOracle`: after the pointer is written, the new oracle must satisfy `!isStale()` and `getPrice() != 0`.

L-8 Insufficient protection against compromised MINT_COMPLETER_ROLE / BURN_COMPLETER_ROLE

LOW

FIXED

DESCRIPTION

The current architecture does not sufficiently limit the damage in the event that operational roles responsible for completing mint and burn requests are compromised: `MINT_COMPLETER_ROLE` and `BURN_COMPLETER_ROLE`.

Both roles can choose the final execution value within `priceDeviationBps` of the oracle-derived price:

- `completeMint()` allows `MINT_COMPLETER_ROLE` to choose `_sharesToMint` at the upper edge of the allowed deviation band;
- `completeBurn()` allows `BURN_COMPLETER_ROLE` to choose `_depositToReturn` at the upper edge of the deviation band and pay funds out of the `treasury`.

As a result, if a completer key is compromised, especially if it is a hot key used by backend/server infrastructure, an attacker or a colluding whitelisted provider can repeatedly extract value from the allowed price deviation without compromising the oracle itself.

On the mint side, the damage is limited by `MintGuard.preMint()` through `mintLimitPerBlock`:

```
mintedPerBlock[block.number] + _amount <= mintLimitPerBlock
```

However, this limit applies only per block. On Ethereum mainnet, blocks are roughly 12 seconds apart, so the attack can be repeated every block up to the configured block cap.

The approximate extract over the incident response window can be estimated as:

```
numberOfBlocks × priceDeviationBps × mintLimitPerBlock
```

For example, if:

```
priceDeviationBps = 1%
mintLimitPerBlock ≈ $2.5M
incident response time ≈ 1 hour ≈ 300 blocks
```

then the potential extract is:

```
300 × 1% × $2.5M = $7.5M
```

If `priceDeviationBps = 10%`, the same scenario scales to approximately `$75M`.

`MintGuard.maxTotalSupply` is not a reliable incident cap. It checks:

```
_currentSupply + _amount <= maxTotalSupply
```

where `_currentSupply` is the current `PermissionedToken.totalSupply()`.

In normal operation, `maxTotalSupply` is typically set above the current outstanding supply to avoid blocking protocol growth. Therefore, it does not define a tight upper bound on losses during a

compromise. Moreover, after large burn operations, `totalSupply` may drop sharply while `maxTotalSupply` remains unchanged. The available mint headroom then becomes:

```
maxTotalSupply - totalSupply
```

and can increase without any admin action. An attacker who can time mint operations around burn activity may obtain a larger available mint limit than before the burn.

The burn side is even less constrained on-chain. `completeBurn()` pays `_depositToReturn` from the `treasury` via:

```
IERC20(depositToken).safeTransferFrom(treasury, provider, _depositToReturn)
```

Unlike mint completion, burn completion has no symmetric on-chain rate limit. The practical ceiling is the treasury balance and the ERC20 allowance granted to `ExternalRequestsManager`. If the allowance is unlimited or too large, a compromised `BURN_COMPLETER_ROLE` can extract value during the undetected compromise window up to:

```
priceDeviationBps × processed burn volume
```

With sufficient burn volume and unlimited allowance, the exposure can approach:

```
priceDeviationBps × totalSupply
```

which may correspond to 1–10% of TVL depending on the configured deviation.

The core issue is not a single missing limit, but that the protocol treats operational completer roles as if their compromise is unlikely or will be stopped immediately. Under a realistic incident model where a hot key may remain compromised for tens of minutes or around an hour, the current controls do not define a small, explicit, and governance-controlled loss budget.

RECOMMENDATION

> We recommend designing the mint/burn architecture under the assumption that `MINT_COMPLETER_ROLE` and `BURN_COMPLETER_ROLE` may be compromised, and that incident response may take a meaningful amount of time, for example around one hour.
>
> For the mint side:
>
> 1. For small / hot mint flows, use a short rolling rate limit, such as an hourly mint limit, so that losses during a compromise window are bounded in advance.
> 2. Route large mint operations that cannot fit into hot-path limits through a separate flow requiring manual multisig approval, not an EOA running on servers.
>
> For the burn side:
>
> 1. For small / hot burn flows, use a short rolling rate limit, such as an hourly burn limit, so that losses during a compromise window are bounded in advance.
> 2. Route large burn operations that cannot fit into hot-path limits through a separate flow requiring manual multisig approval, not an EOA running on servers.
>
> Alternative approaches for the mint side:
>
> 1. Introduce a separate incident budget, such as `maxAvailableSharesToMint`, set by multisig

rather than an operational EOA.
2. Decrease this budget on every successful mint.
3. Once the budget reaches zero, `preMint()` should revert until multisig explicitly restores the limit after verifying that monitoring does not indicate an ongoing attack.
4. Keep `maxTotalSupply` only as a long-term supply ceiling, not as the primary incident cap.

Alternative approaches for the burn side:

1. Use treasury allowance as an explicit incident throttle:
   - do not grant unlimited allowance;
   - keep the allowance from treasury to `ExternalRequestsManager` intentionally small;
   - allow exposure to decrease as burns are completed;
   - increase allowance only through multisig after checking monitoring.
2. Consider a separate `maxAvailableDepositToReturn`, decremented on each burn completion and restored only by multisig.

We also recommend configuring monitoring and alerts for completions near the upper edge of `priceDeviationBps`.

The objective is to ensure that compromise of a hot completer/burner roles does not become open access to `priceDeviationBps`-based extraction across the protocol's available volume.

CLIENT'S COMMENTARY

Partially Fixed (mint) — `8482b55`; Acknowledged (burn)
`MintGuard` now enforces a rolling 1-hour window cap = `max(supply × mintLimitPerWindowBps / 10_000, mintLimitFloor)`. With suggested params (`bps=100`, `floor=$1M`, `window=1h`) the per-hour cycle extract on a $100M-supply product at 0.5% deviation drops from ~$14.85M to ~$10K — ~1500× reduction.
Burn side: Sustained extraction via burn requires shares to burn, and minting them is now bound by `MintGuard`'s per-window cap — so the mint->burn cycle profit is bounded by the mint side. Pure burn drain is limited to the pre-existing balance of a colluding KYC'd provider (one-shot, not repeatable), further capped by the bounded treasury allowance to ERM which is refreshed only via multi-party approval. Net economic upside of attacking the burn path on top of `MintGuard` is negligible.

L-9 Oracle rotation — no on-chain check that new price is within an acceptable band of the old oracle

LOW

FIXED

DESCRIPTION

`ExternalRequestsManager.setOracle()` and `MintGuard.setOracle()` validate structural compatibility but never compare the new oracle's spot price to the price returned by the oracle being replaced.

RECOMMENDATION

> We recommend checking that the new oracle's price is within an acceptable band of the old oracle's price.

CLIENT'S COMMENTARY

Fixed — `2543fb2`
`ERM.setOracle` reuses `priceDeviationBps` via `_assertWithinDeviation(newPrice, oldPrice)`. `MintGuard.setOracle` enforces an independently configurable `oracleRotationDeviationBps`

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL	AUDITS DELIVERED	WEB3 PROTOCOLS
$50B+	300+	80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io